Estadística de palabras II

wordstandarddeviation

multifilewc

Equipo 2 — Jeshua Vera Flores, Andrea Isabel Vega Martínez

Prof. José Javier Moya Moya

count	116
length	513
square	2883

Tres renglones.
Eso es *todo* lo que
devuelve el trabajo.

Nos tocaron dos ejemplos que enseñan cosas distintas

`wordstandarddeviation`

Mide qué tan dispares son las longitudes de las palabras de un texto.

La lección: qué cosas puede calcular MapReduce… y cuáles no, aunque te las devuelva.

`multifilewc`

Cuenta palabras. Exactamente igual que el `wordcount` que ya vieron.

La lección: cómo Hadoop reparte el trabajo, y por qué eso importa más que el cálculo.

Los dos corren sobre el contenedor de la práctica, en modo standalone, sin HDFS ni YARN.

El molde que ya conocen

Todo ejemplo de MapReduce cabe en tres casillas. Las de `wordcount`:

Map	Shuffle	Reduce

`"la fase map"`
`↓`
`(la, 1) (fase, 1) (map, 1)`
	`Hadoop agrupa`
	`por clave`
	`la → [1, 1]`
		`suma la lista`
		`↓`
		`la  2`

Guárdense esto

El Reduce **solo sabe juntar una lista en un número**. Nada más. Con eso basta para los dos ejemplos.

Qué tan parejas son las palabras

Cuenta las letras de cada palabra y mide qué tan dispersas están esas longitudes. El promedio no basta: estos dos renglones tienen el mismo promedio.





La desviación chica dice que casi todas las palabras miden lo mismo. La grande, que hay cortas mezcladas con largas.

Para qué sirve eso en la vida real

Medir qué tan difícil es un texto

La longitud de las palabras es una de las señales que usan los índices de legibilidad, como el de Fernández-Huerta para el español. Editoriales, periódicos y apps educativas la usan para etiquetar un texto como fácil o difícil.

Detectar texto basura

El texto escrito por una persona tiene una distribución de longitudes bastante estable. Cuando esa distribución se sale de rango, suele ser spam, relleno para SEO o salida de un bot.

¿Y por qué MapReduce? Con un texto cabe en Python. Con **50 millones de artículos** no cabe en una computadora, y ahí es donde tiene sentido repartir el conteo entre muchas.

El Mapper emite tres pares por cada palabra

No uno. Tres. Y siempre las mismas tres claves, sin importar cuál sea la palabra.

`dato` 4 letras

emite

length	**4**
square	**16**
count	**1**

- `length` — cuántas letras tiene
- `square` — esas letras **al cuadrado**
- `count` — siempre `1`, para ir contando palabras

¿Y para qué quiere el cuadrado?
Esa es la parte bonita. Aguanten dos diapositivas.

El recorrido completo

Con una línea de cuatro palabras, para que se vea entero.

Entrada

```
el dato es el
```

↓

Map

```
el(2)   → (length, 2) (square,  4) (count, 1)
dato(4) → (length, 4) (square, 16) (count, 1)
es(2)   → (length, 2) (square,  4) (count, 1)
el(2)   → (length, 2) (square,  4) (count, 1)
```

↓

Shuffle

```
count  → [1, 1, 1, 1]
length → [2, 4, 2, 2]
square → [4, 16, 4, 4]    solo tres claves, siempre
```

↓

Reduce

```
suma cada lista  →  count 4    length 10    square 28
```

Y aquí **se acaba** el trabajo de MapReduce. Tres números escritos en `part-r-00000`.

MapReduce nunca calculó la desviación

El job terminó con tres sumas. Después, ya cerrado el trabajo, el programa **vuelve a abrir** `part-r-00000`, lee los tres números y hace la división y la raíz él solito.

```
media  = 10 / 4 = 2.5
var    = 28/4 − 2.5² = 0.75
stddev = √0.75 = 0.8660254…
```

¿Por qué así? Una suma se puede partir entre mil máquinas y juntar después. Una raíz cuadrada sobre un total global, no. Y tampoco hace falta: es una sola operación sobre tres numeritos.

Por eso el cuadrado

Por una identidad de estadística:

$$Var(X) = E[X^2] - (E[X])^2$$

Guardando la suma de longitudes y la suma de longitudes al cuadrado, se reconstruye la varianza **sin volver a leer el texto**. Por eso el Mapper carga el cuadrado desde el principio.

Ojo: divide entre n, no entre n−1.
Es la desviación poblacional.

Compruébenlo a mano

datos/mini.txt — 16 palabras, sin puntuación a propósito

```
el dato es el nuevo petroleo
los datos crudos no sirven
hay que limpiar los datos
```

part-r-00000

count	**16**
length	**66**
square	**328**

```
media  = 66 / 16
       = 4.125

var    = 328/16 - 4.125²
       = 20.5 - 17.015625
       = 3.484375
```

lo que imprime Hadoop

```
The standard deviation is:
1.8666480653835098
```

Cuadra

multifilewc es wordcount. Literal.

Mismo Mapper, mismo Reducer, misma salida. Lo comprobamos con `diff`.

	wordcount	multifilewc
Mapper	palabra → 1	palabra → 1
Reducer	IntSumReducer	IntSumReducer
Salida	74 palabras	74 palabras
InputFormat	**TextInputFormat**	**CombineFileInputFormat**

Un solo renglón distinto. Y ese renglón **no cambia qué se calcula**, cambia **cómo se reparte el trabajo**.

Lo que pasa antes del Map

El InputFormat es la parte de Hadoop que decide **cómo se parte la entrada en unidades de trabajo**. A cada unidad se le llama *split*, y vale la regla:

un split = una tarea map

- `TextInputFormat` hace **mínimo un split por archivo**.
- `CombineFileInputFormat` **mete varios archivos chicos en un solo split**.

La única línea que cambia en el ejemplo

```
job.setInputFormatClass(
    MyInputFormat.class);
```

donde `MyInputFormat` hereda de `CombineFileInputFormat`.

Detalle fino

La clave que recibe el Mapper ya no es el desplazamiento en bytes, como en `wordcount`. Es un `WordOffset`: **nombre de archivo + posición**. Como el split ahora mezcla varios archivos, hay que saber de cuál venía cada línea.

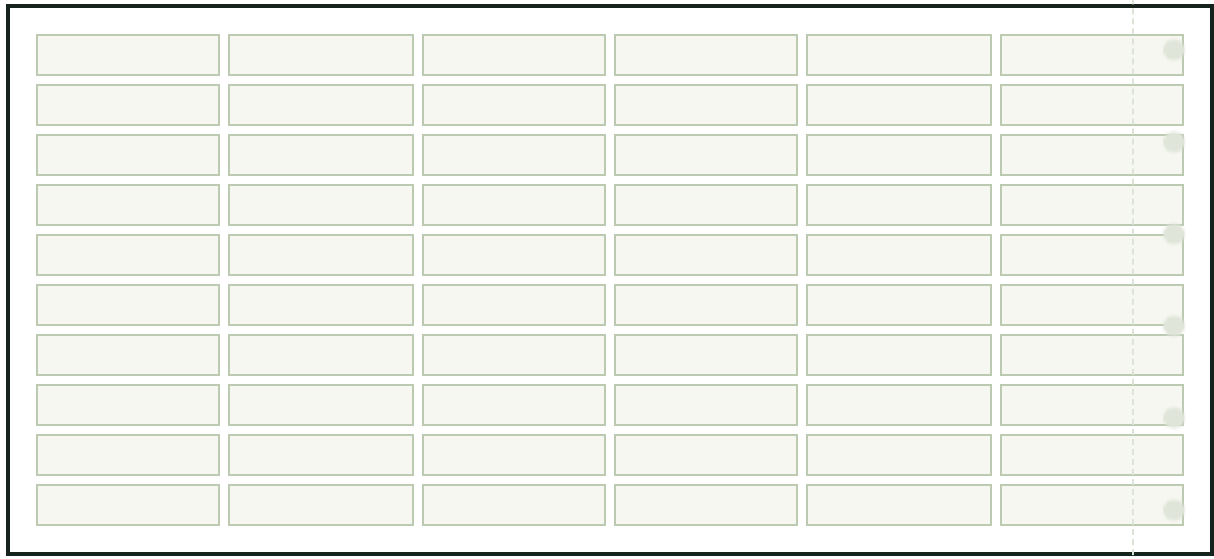
Sesenta sobres, o una caja

Los mismos 60 archivos de nuestra demo, repartidos de las dos maneras.



wordcount

`60 splits → 60 tareas map`



multifilewc

`1 split → 1 tarea map`

Las mismas hojas y el mismo destino. Lo que cambia es cuántas veces pagas el envío.

El problema de los archivos pequeños

Pasa todo el tiempo: logs con un archivo por hora por servidor, lecturas de sensores, exportaciones de APIs.

100 MB → **50 000**

50 000 archivos de 2 KB — tareas map con wordcount

El costo no está en los datos, está en las tareas.

- Cada tarea arranca su propia JVM: segundos en levantarla, milisegundos en procesar 2 KB.
- El ApplicationMaster tiene que pedir, agendar y vigilar 50 000 contenedores.
- En HDFS cada archivo ocupa memoria del NameNode, pese lo que pese.

Con splits de 128 MB: de 50 000 tareas a 1.

Demo en vivo

Todo dentro del contenedor, como usuario `hadoop`. Las rutas completas están en el README.

Parte A — la desviación

```
hadoop jar $JAR wordstandarddeviation \
  datos/mini.txt \
  salida-stddev-mini

cat salida-stddev-mini/part-r-00000
```

count	**16**
length	**66**
square	**328**

Parte B — la comparación

```
scripts/generar_muchos_archivos.sh 60

hadoop jar $JAR wordcount \
  datos/muchos salida-wc 2>&1 | tee wc.log

hadoop jar $JAR multifilewc \
  datos/muchos salida-mfwc 2>&1 | tee mfwc.log

grep 'number of splits' wc.log mfwc.log
```

Un atajo: `scripts/ejecutar.sh` corre todo de un jalón.

El momento de la verdad

Los mismos 60 archivos, los mismos 240 KB, los dos ejemplos.

```
wordcount      number of splits: 60
multifilewc    number of splits: 1
```

```
diff salida-wc/part-r-00000 salida-mfwc/part-r-00000
```

IDÉNTICOS

Mismo resultado. **Sesenta veces menos tareas.**

Su turno

datos/frase.txt

```
big data no es solo mucho dato
```

Llenen `count`, `length` y `square` a mano, saquen la desviación, y después córranlo en Hadoop para ver si les cuadra.

`count`	?
`length`	?
`square`	?

- ¿Por qué la salida tiene **tres renglones** aunque el texto crezca? ¿Y quién calcula la raíz cuadrada?
- Generen **30** archivos y cuenten los splits de cada ejemplo. ¿Las salidas son iguales?
- Si tuvieran **50 000** archivos de 2 KB, ¿cuántas tareas map levantaría `wordcount`, y por qué es un problema?
- Reto: `multifilewc` juntó todo en un solo split. Con 50 000 archivos eso también estaría mal. ¿Qué propiedad le pondrían para limitar el tamaño del split?

Enunciado y solución en `ejercicio/` del repositorio.

Lo que se llevan

- **MapReduce solo sabe sumar.** Todo lo que no sea repartible se hace afuera, al final, sobre los totales.
- **Cómo se reparte el trabajo pesa tanto como qué se calcula.** Un InputFormat distinto bajó 60 tareas a 1 sin tocar una línea del cálculo.

---

github.com/JVeFloo/
bda-equipo-2-estadistica-de-palabras-ii

count	**116**
length	**513**
square	**2883**

116 palabras.
513 letras entre todas.
Y el cuadrado, que es lo que deja reconstruir la varianza.

σ = 2.3012397785607734